

Đồ án 2 - Khai thác tập phổ biến

Thuật toán FP-Max

(Maximal Frequent Itemset Mining)

Môn học: CSC14004 - Khai thác dữ liệu và ứng dụng

Sinh viên thực hiện:

22120457 Khưu Hải Châu
23122006 Lưu Thượng Hồng
23122020 Nguyễn Thiên Ấn
23122034 Lê Nguyên Khang
23122036 Nguyễn Ngọc Khoa



GVHD:

GS.TS. Lê Hoài Bắc
ThS. Lê Nhựt Nam

Học kỳ: 2 - 2025/2026

Nội dung

- 1 Đặt vấn đề
- 2 Thuật toán FP-Max
- 3 Ví dụ minh họa
- 4 Cài đặt
- 5 Thực nghiệm
- 6 Ứng dụng
- 7 Kết luận

Bài toán Frequent Itemset Mining (FIM)

- CSDL transaction $D = \{T_1, \dots, T_n\}$, $T_j \subseteq I$ (I : tập item).
- Độ hỗ trợ: $\text{sup}(X) = |\{T_j \in D \mid X \subseteq T_j\}|$.
- X **phổ biến (frequent)** nếu $\text{sup}(X) \geq \theta \cdot |D|$ (minsup).
- **Tập đóng (closed)**: không có superset phổ biến cùng support.
- **Tập tối đại (maximal)**: không có superset nào phổ biến.

frequent $\supseteq$ closed $\supseteq$ maximal

Tính chất Apriori (downward closure)

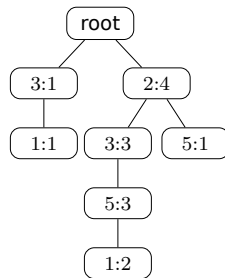
$X \subseteq Y \Rightarrow \text{sup}(Y) \leq \text{sup}(X)$ — cơ sở để mọi thuật toán FIM tĩa không gian tìm kiếm.

Vì sao maximal?

Biết toàn bộ maximal itemset là đủ để biết *mọi* itemset có phổ biến hay không — biểu diễn cô đọng nhất của không gian phổ biến.

FP-tree – cấu trúc nền cho FP-Max

- Apriori: sinh ứng viên theo mức, quét CSDL nhiều lần.
- FP-Growth: chỉ quét **2 lần**, nén transaction vào **FP-tree**, khai thác bằng *conditional pattern base* – không sinh ứng viên.
- Node: item, count, parent, children.
Header table: item $\rightarrow$ list node.
- Chèn cây theo support **giảm dần** $\Rightarrow$ tiền tố chung được gộp.
- FP-Max **tái sử dụng đúng** FP-tree và cơ chế này của FP-Growth.



FP-tree của CSDL toy, $\theta = 0.6$

Ý tưởng cốt lõi của FP-Max

- Khác FP-Growth: **không** xuất mọi tập phổ biến, chỉ giữ tập **tối đại**.

Danh sách MFI (maximal frequent itemset list)

Candidate mới bị *bỏ qua* nếu là tập con của một MFI sẵn có; ngược lại, mọi MFI cũ là con của candidate bị loại rồi thêm candidate vào.

Bất biến: không có hai phần tử nào lồng nhau.

Tỉa theo superset

Tại head hiện tại, gọi *tail* = item còn phổ biến trong conditional pattern base. Nếu $head \cup tail \subseteq$ một MFI có sẵn $\Rightarrow$ **cắt cả nhánh**, không cần dựng conditional FP-tree.

- Duyệt header theo support **tăng dần** $\Rightarrow$ MFI lớn nhanh, tỉa phát huy sớm.
- **Single-path shortcut**: cây co thành 1 đường $\Rightarrow$ sinh thẳng 1 candidate, không đệ quy.

Require: FP-tree, *head* hiện tại, danh sách MFI

- 1: **if** FP-tree chỉ có một đường đi **then** ▷ tốt nhất: 1 candidate duy nhất
- 2: *candidate* $\leftarrow$ *head* $\cup$ item trên đường đi
- 3: Cập nhật MFI với *candidate* (insert_mfi!)
- 4: **return**
- 5: **end if**
- 6: Sắp item trong header theo support **tăng dần**
- 7: **for** mỗi item *a* trong header **do**
- 8: *head* $\leftarrow$ *head* $\cup$ {*a*}; tạo conditional pattern base và *tail*
- 9: **if** *head* $\cup$ *tail* $\not\subseteq$ MFI nào **then** ▷ superset pruning
- 10: Dựng conditional FP-tree, đệ quy với *head* mới
- 11: **end if**
- 12: *head* $\leftarrow$ *head* $\setminus$ {*a*}
- 13: **end for**

Trường hợp	Xây cây	Khai thác	Không gian
Tốt nhất (single-path)		$O(\bar{\ell})$	$O(\bar{\ell})$
Trung bình	$O(L \log \bar{\ell})$	$O(M \cdot c)$	$O(N + M\bar{\ell})$
Xấu nhất		$O(2^m)$	$O(L + M\bar{\ell})$

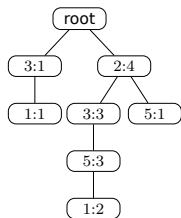
Yếu tố ảnh hưởng chính:

- Số tập tối đại M – tăng nhanh khi minsup giảm / dữ liệu dày.
- Độ dài transaction trung bình $\bar{\ell}$ – transaction dài $\Rightarrow$ cây sâu hơn.
- Mức độ chia sẻ tiền tố – quyết định độ nén của FP-tree.

Ví dụ 1: CSDL cơ bản

Transaction	Item
T_1	{1, 3, 4}
T_2	{2, 3, 5}
T_3	{1, 2, 3, 5}
T_4	{2, 5}
T_5	{1, 2, 3, 5}

$\theta = 0.6 \Rightarrow \text{minsup} = 3; 2 \prec 3 \prec 5 \prec 1$



Head	Cond. base	Tail	Candidate	MFI
1	(3):1, (2,3,5):2	{3}	{1,3}	{1,3}
2	$\emptyset$	$\emptyset$	{2}	+{2}
3	(2):3	{2}	{2,3}	-{2}+{2,3}
5	(2,3):3,(2):1	{2,3}	{2,3,5}	-{2,3}+{2,3,5}

Kết quả

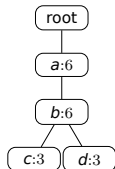
{1, 3}:3, {2, 3, 5}:3

Ví dụ 2: Tỉa theo superset

Transaction	Item
T_1-T_3	$\{a, b, c\}$
T_4-T_6	$\{a, b, d\}$

$\theta = 0.5 \Rightarrow \text{minsup} = 3$; header

$c \prec d \prec a \prec b$



Head	$head \cup tail$	Hành động	MFI
c	$\{a, b, c\}$	dựng cây	$+\{a, b, c\}$
d	$\{a, b, d\}$	dựng cây	$+\{a, b, d\}$
a	$\{a\}$	$\subseteq \{a, b, c\}$	tỉa
b	$\{a, b\}$	$\subseteq \{a, b, c\}$	tỉa

Kết quả

$\{a, b, c\}:3, \{a, b, d\}:3$

So sánh

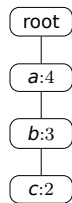
FP-Growth phải xuất **10** tập phổ biến; FP-Max chỉ giữ **2** tối đại nhờ tỉa 2 nhánh cuối.

Ví dụ 3: FP-tree single-path

Transaction	Item
T_1	$\{a, b, c\}$
T_2	$\{a, b, c\}$
T_3	$\{a, b\}$
T_4	$\{a\}$

$$\theta = 0.5 \Rightarrow \text{minsup} = 2$$

$\{a\} \subset \{a, b\} \subset \{a, b, c\}$ (lồng nhau)



So sánh đầu ra

Thuật toán	Đầu ra
FP-Growth	$\{a\}, \{b\}, \{c\}, \{a, b\}, \{a, c\}, \{b, c\}, \{a, b, c\}$ $\Rightarrow 2^3 - 1 = 7$ tập
FP-Max	$\{a, b, c\}:2 \Rightarrow$ đúng 1 tối đại

Lỗi tắt single-path

Cây co thành một đường $\Rightarrow$ FP-Max phát hiện ngay tại lời gọi gốc, gom mọi item thành **một** candidate (support = count nhỏ nhất), không đệ quy. Đường dài k : FP-Growth sinh $2^k - 1$ tập, FP-Max chỉ 1 - nguồn hiệu năng chính.

Cài đặt

- **Ngôn ngữ:** Julia ≥ 1.9 (đúng ưu tiên đề bài) – package `FrequentItemsetMining`.
- **Thành phần:** `structures.jl`, `data_loader.jl`,
`algorithm/{fp_growth, fp_growth_opt, fp_max}.jl`, `association_rules.jl`, `io.jl`.
- **2 chiến lược khai thác tối đại** để đo tác động của tĩa:
 - base: naive-maximal – mine-all bằng FP-Growth rồi lọc tối đại.
 - opt: FP-Max trực tiếp – danh sách MFI + superset pruning.
- **Tối ưu FP-Growth nền:** integer encoding, header/children `Dict` typed, single-path shortcut, `@inbounds` trong hot loop (type-stable, kiểm bằng `@code_warntype`).
- **Kiểm thử:** `julia --project=. test/runtests.jl` – parser, FP-Growth, FP-Max trên CSDL toy, đối chiếu base VS opt.

Chạy chương trình

```
julia --project=. main.jl <input> <minsup> [fp-max|...] [output]
```

Tính đúng đắn & thời gian chạy theo minsup

Đúng đắn

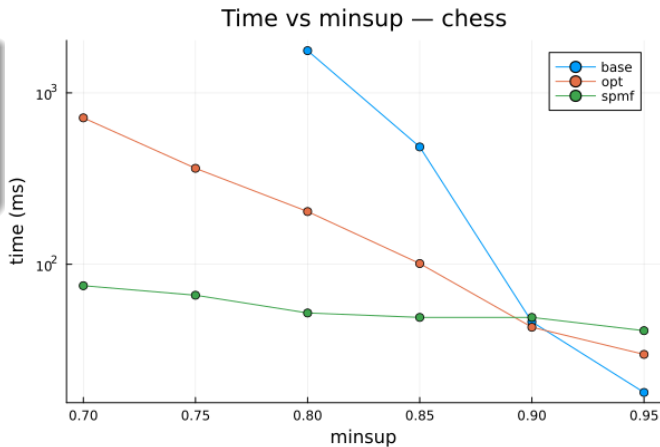
match_ratio = 1.0, support_mismatch = 0 so với SPMF FPMMax trên **mọi** cấu hình kiểm tra (Chess, Mushrooms, Retail, T10I4D100K, Accidents).

Chess, minsup 0.80:

opt nhanh hơn base $\approx 8.68\times$ (203 ms vs 1763 ms).

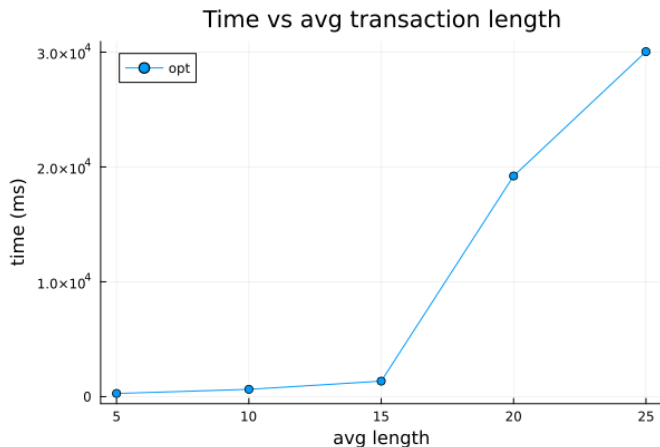
Minsup thấp hơn (0.70): base **skip** (bùng nổ), opt vẫn chạy.

Trên dữ liệu **thừa** (Retail, T10I4D100K): xu hướng ngược lại - tải ít tác dụng, opt chậm hơn base.



Thời gian chạy theo minsup trên Chess

Bộ nhớ, scalability & độ dài transaction



Thời gian chạy khi tăng độ dài transaction trung bình

Bộ nhớ (Chess 0.85): base 119.71MB $\rightarrow$ opt 25.23MB ($4.75\times$ ít hơn). Trên dữ liệu thưa, opt cấp phát *nhiều hơn* base.

Scalability (Accidents, $\theta=0.7$): 34k $\rightarrow$ 340k transaction: 690 $\rightarrow$ 7673 ms ($\approx 11.1\times$) - gần tuyến tính.

Độ dài transaction: AvgLen 15 $\rightarrow$ 20: #MFI nhảy 100 $\rightarrow$ 4,950, thời gian nhảy 1.3s $\rightarrow$ 19.2s - đúng cận mũ trường hợp xấu.

Ứng dụng: Market Basket Analysis (Groceries)

- Groceries: 9,835 transaction, 169 mặt hàng, AvgLen 4.41.
- minsup = 0.01, minconf = 0.2 $\Rightarrow$ 333 frequent itemsets, 231 luật (lift > 1).
- Luật sinh bằng bộ máy FP-Growth đầy đủ (FP-Max chỉ cho tối đại, không giữ support tập con cần để tính confidence/lift).

Vế trái $\Rightarrow$ Vế phải		Sup	Conf	Lift
citrus fruit + other veg.	$\Rightarrow$ root veg.	.0104	.359	3.295
other veg. + yogurt	$\Rightarrow$ whipped/sour cream	.0102	.234	3.267
other veg. + tropical fruit	$\Rightarrow$ root veg.	.0123	.343	3.145
beef	$\Rightarrow$ root veg.	.0174	.331	3.040
citrus fruit + root veg.	$\Rightarrow$ other veg.	.0104	.586	3.030

Ý nghĩa: gợi ý mua kèm rau-củ-trái cây, thiết kế combo nguyên liệu (beef + root vegetables), bố trí kệ hàng theo nhóm đồng mua.

- Cài đặt FP-Max bằng Julia, đúng **100%** so với SPMF FPMax trên mọi benchmark kiểm tra.
- **Điểm mạnh:** chạy được ở các vùng baseline naive-maximal bùng nổ (Accidents, minsup thấp trên dữ liệu dày); nhanh và tiết kiệm bộ nhớ hơn baseline trên dữ liệu dày.
- **Điểm yếu:** chậm hơn baseline trên dữ liệu thưa (tỉa ít tác dụng, chi phí cài đặt cao); chậm hơn SPMF (Java tối ưu lâu năm) trên đa số cấu hình.
- **Hướng phát triển:** cây mã hoá Int thay String, mang support theo trong khai thác để bỏ bước quét lại CSDL, MFI-tree thay danh sách tuyến tính, song song hoá nhánh độc lập.

Cảm ơn Thầy và các bạn đã lắng nghe!

Q&A